

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра информационной безопасности

ОТЧЁТ
по лабораторной работе №3
по дисциплине: «Компьютерная графика»
Тема: «Формирование различных поверхностей с использованием
пространственного разворота и ортогонального проецирования на
плоскость при визуализации»

Студент гр. 4364	_____	Ефремов Э. А.
Студент гр. 4364	_____	Поляков Н. Н.
Преподаватель	_____	Матвеева И. В.

Санкт-Петербург
2026

Вариант № 3 (тема 3). Сформировать поверхность Безье для различного задающего многогранника. Обеспечить поворот сформированной поверхности вокруг осей X и Y.

ЗАДАНИЕ

Разработать программный продукт на языке высокого уровня, реализующий формирование и пространственную визуализацию поверхности Безье по произвольному задающему многограннику (сетке контрольных точек). Предусмотреть:

- задание размеров задающего многогранника (от 2×2 до 8×8) и редактирование координат X, Y, Z каждой контрольной точки;
- визуализацию поверхности Безье и самого задающего многогранника (характеристической сетки);
- поворот сформированной поверхности и контрольной сетки вокруг осей X и Y с возможностью плавного изменения углов;
- ортогональное проецирование трёхмерной сцены на плоскость визуализации с автоматическим масштабированием;
- сохранение и загрузку набора контрольных точек во внешний файл;
- отображение исходных данных и результатов в графическом интерфейсе пользователя.

Специализированные графические библиотеки типа OpenGL не использовать.

ЦЕЛЬ РАБОТЫ

Изучить математический аппарат формирования параметрических поверхностей в компьютерной графике – тензорное произведение базисных полиномов Бернштейна (поверхность Безье); освоить пространственные аффинные преобразования трёхмерных объектов (поворот вокруг осей X и Y) и метод ортогонального проецирования на плоскость визуализации; реализовать соответствующий алгоритм и интерактивный интерфейс

пользователя для управления задающим многогранником и параметрами разворота сцены.

МАТЕМАТИЧЕСКАЯ ОСНОВА

1. Сплайновые представления и базисные полиномы Бернштейна

Сплайновая поверхность в компьютерной графике описывается двумя наборами ортогональных сплайновых кривых и задаётся набором контрольных точек, определяющих общую форму объекта. Если полиномиальные участки не проходят через внутренние контрольные точки, а лишь приближаются к ним, речь идёт об аппроксимации. Кривые и поверхности Безье относятся именно к аппроксимирующим сплайнам: они интерполируют только угловые (крайние) точки задания и целиком лежат внутри выпуклой оболочки задающего многогранника.

Основой описания служат базисные полиномы Бернштейна степени n :

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad \binom{n}{i} = \frac{n!}{i!(n-i)!} \quad (1)$$

где $C(n, i)$ – число сочетаний из n по i . На отрезке $t \in [0, 1]$ полиномы неотрицательны и образуют единичное разбиение:

$$\sum_{i=0}^n B_{i,n}(t) \equiv 1, \quad t \in [0, 1] \quad (2)$$

Свойство (2) обеспечивает инвариантность формы поверхности относительно аффинных преобразований системы координат и устойчивость вычислений: сумма весов всех контрольных точек при любых u и v строго равна единице.

2. Поверхность Безье как тензорное произведение

В работе реализован непараметрический (тензорный) вариант поверхности Безье. По задающей матрице контрольных точек $P[i, j]$, $i = 0 \dots m$, $j = 0 \dots n$, точками которой являются трёхмерные векторы, поверхность определяется двойной суммой:

$$S(u, v) = \sum_{i=0}^m \sum_{j=0}^n B_{i,m}(u) B_{j,n}(v) P_{i,j}, \quad 0 \leq u \leq 1, \quad 0 \leq v \leq 1 \quad (3)$$

Здесь u – параметр, изменяющийся вдоль строк задающего многогранника, v – вдоль столбцов. Степени полиномов определяются размерами сетки:

$$m = N_r - 1, \quad n = N_c - 1 \quad (4)$$

где $N(r)$ – число строк, $N(c)$ – число столбцов матрицы контрольных точек. Благодаря тензорной структуре формула (3) разделяется на два последовательных суммирования:

$$S(u, v) = \sum_{j=0}^n \left(\sum_{i=0}^m B_{i,m}(u) P_{i,j} \right) B_{j,n}(v) \quad (5)$$

то есть поверхность может быть построена как семейство кривых Безье по параметру u , которые затем «сшиваются» по параметру v . Это позволяет сократить число операций, однако при размерах сетки до 8×8 и числе сегментов табуляции до 60 прямой двойной цикл (реализованный в программе) требует не более $61 \cdot 61 \cdot 64 \approx 240$ тысяч операций умножения на кадр, что обеспечивает перерисовку сцены в реальном времени.

Из (3) следуют важные для контроля работы программы свойства. Поверхность интерполирует четыре угловые точки задающего многогранника:

$$S(0, 0) = P_{0,0}, \quad S(0, 1) = P_{0,n}, \quad S(1, 0) = P_{m,0}, \quad S(1, 1) = P_{m,n} \quad (6)$$

и целиком лежит в выпуклой оболочке контрольных точек, то есть её координаты не могут выйти за пределы габаритов задающего многогранника:

$$\min_{i,j} P_{i,j} \leq S(u, v) \leq \max_{i,j} P_{i,j} \quad (7)$$

Частным случаем является сетка 2×2 ($m = n = 1$): поверхность Безье вырождается в билинейную (линейчатую) поверхность, для которой формула (3) принимает вид

$$Q(u, v) = P_{0,0}(1-u)(1-v) + P_{0,1}(1-u)v + P_{1,0}u(1-v) + P_{1,1}uv \quad (8)$$

что полностью совпадает с классическим описанием билинейной поверхности через четыре угловые точки. Обе зависимости – (3) и (8) – проверены численно.

3. Дискретизация (табуляция) поверхности

Визуализация поверхности выполняется в виде каркаса: параметры u и v пробегают равномерную сетку значений с числом сегментов N :

$$u_i = \frac{i}{N}, \quad v_j = \frac{j}{N}, \quad i, j = 0, 1, \dots, N \quad (9)$$

Для каждой пары $(u(i), v(j))$ по формуле (3) вычисляется пространственная точка поверхности, после чего соседние точки соединяются отрезками прямых. Полученная сетка линий является линейной аппроксимацией поверхности; при увеличении N ломаная приближается к точной криволинейной поверхности (погрешность аппроксимации имеет порядок $O(1/N^2)$). В программе число сегментов регулируется в диапазоне 4...60, по умолчанию $N = 20$.

4. Поворот поверхности вокруг осей X и Y

Все преобразования трёхмерных объектов выполнены в соответствии с матричным аппаратом аффинных преобразований. В однородных координатах точка имеет вид $(x, y, z, 1)$, а произвольное аффинное преобразование описывается матрицей 4×4 , подматрицы которой отвечают за масштабирование, поворот, перенос и перспективные преобразования. В работе используются только повороты, поэтому достаточно матриц 3×3 .

Матрица поворота вокруг оси X на угол α :

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix} \quad (10)$$

Матрица поворота вокруг оси Y на угол β :

$$R_y(\beta) = \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix} \quad (11)$$

Координаты точки преобразуются умножением матрицы на вектор-столбец:

$$P' = R P, \quad P = (x, y, z)^T \quad (12)$$

В программе повороты выполняются последовательно: сначала вокруг оси X, затем вокруг оси Y, что эквивалентно умножению на композицию матриц

$$R = R_y(\beta) R_x(\alpha) = \begin{pmatrix} \cos \beta & \sin \beta \sin \alpha & \sin \beta \cos \alpha \\ 0 & \cos \alpha & -\sin \alpha \\ -\sin \beta & \cos \beta \sin \alpha & \cos \beta \cos \alpha \end{pmatrix} \quad (13)$$

В однородных координатах та же композиция записывается матрицей 4×4 (перенос и перспектива отсутствуют, s = 1):

$$M = \begin{pmatrix} \cos \beta & \sin \beta \sin \alpha & \sin \beta \cos \alpha & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ -\sin \beta & \cos \beta \sin \alpha & \cos \beta \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (14)$$

Матрицы поворота ортогональны, их определитель равен единице, поэтому преобразование является движением: длины отрезков, углы и форма задающего многогранника сохраняются. Например, при $\alpha = 90^\circ$ поворот вокруг оси X переводит точку (x, y, z) в

$$\alpha = 90^\circ : (x, y, z) \longrightarrow (x, -z, y) \quad (15)$$

Поворот применяется как к точкам поверхности, так и к контрольным точкам, поэтому задающий многогранник и построенная по нему поверхность разворачиваются согласованно.

5. Ортогональное проецирование на плоскость визуализации

После поворота выполняется ортогональное (параллельное) проецирование трёхмерной сцены на плоскость экрана. Проекция точки (x, y, z) в пиксельные координаты холста шириной W и высотой H имеет вид:

$$x_{\text{px}} = \frac{W}{2} + k x, \quad y_{\text{px}} = \frac{H}{2} - k y \quad (16)$$

где k – коэффициент масштаба, а знак «минус» во второй формуле учитывает, что ось Y на экране направлена вниз. При ортогональном

проецировании координата z не влияет на положение точки на экране: проецирование выполняется вдоль оси Z , поэтому изображение является каркасным (проволочным). Удаление невидимых линий и раскраска поверхностей относятся к темам 5 и 6 и в данной работе не выполняются.

Коэффициент масштаба вычисляется автоматически по габаритам повернутого задающего многогранника:

$$k = \frac{0,72 \min(W, H)}{\text{span}}, \quad \text{span} = \max(\Delta x, \Delta y, \Delta z, 1) \quad (17)$$

где

$$\Delta x = x_{\max} - x_{\min}, \quad \Delta y = y_{\max} - y_{\min}, \quad \Delta z = z_{\max} - z_{\min} \quad (18)$$

Множитель 0,72 оставляет поля по краям холста, а коэффициент k одинаков для обеих осей, поэтому пропорции изображения не искажаются. При отключённом автомасштабе используется фиксированное значение k , задаваемое пользователем.

ХОД РАБОТЫ

1. Средства реализации

Программа реализована на языке Python. Использована только стандартная библиотека языка: `math` (вычисление сочетаний и тригонометрических функций), `tkinter` и `ttk` (графический интерфейс пользователя, холст, элементы управления), `dataclasses` (описание точки пространства). Сторонние библиотеки не применяются: файл `requirements.txt` не содержит зависимостей, установка пакетов не требуется. Специализированные графические библиотеки типа OpenGL не использовались — вся растровая визуализация выполняется штатными примитивами холста Tk (`create_line`, `create_oval`, `create_text`).

2. Структура программы и ключевые функции

Программа состоит из математического ядра и класса графического интерфейса. Ядро не зависит от интерфейса и может тестироваться отдельно. Состав функций и методов приведён в таблице 1.

Таблица 1. Функции и методы программы

Имя	Назначение
Point3D	точка пространства (x, y, z); определена операция сложения и умножения на скаляр для вычисления линейных комбинаций
binomial(n, k)	биномиальный коэффициент $C(n, k)$
bernstein(n, i, t)	базисный полином Бернштейна $B(i, n, t)$ по формуле (1)
bezier_surface(control_points, u, v)	точка поверхности по формуле (3) для произвольной сетки контрольных точек
sample_surface(control_points, steps)	табуляция поверхности в сетку $(steps+1) \times (steps+1)$ точек
rotate_point(p, angle_x, angle_y)	поворот точки – композиция матриц (13)
project_orthographic(p, W, H, scale)	ортогональная проекция на холст по (16)
bounds(points)	габариты набора точек для автомасштаба (18)
BezierSurfaceApp	главное окно: панель управления, редактор контрольных точек, холст, статусная строка
_dimensions_changed, _load_example	изменение размеров задающего многогранника и загрузка примера 3×4
_read_editor, _rebuild_editor	чтение и построение таблицы координат контрольных точек
_save_points, _load_points	обмен набором контрольных точек с текстовым файлом
_get_scale	вычисление коэффициента масштаба (17) с учётом автомасштаба
redraw	полная перерисовка сцены (поворот, табуляция, проекция, вывод)
_draw_axes, _draw_control_polyhedron, _draw_surface	отрисовка осей, задающего многогранника и поверхности

Ниже приведены листинги ключевых функций математического ядра.

Вычисление базисного полинома Бернштейна (формула 1):

```
def binomial(n: int, k: int) -> int:
    if k < 0 or k > n:
        return 0
    return math.comb(n, k)

def bernstein(n: int, i: int, t: float) -> float:
    return binomial(n, i) * (t ** i) * ((1.0 - t) ** (n - i))
```

Построение точки поверхности Безье (формула 3) для произвольного задающего многогранника:

```
def bezier_surface(control_points, u: float, v: float) -> Point3D:
```

```

    """Tensor-product Bezier surface for an arbitrary m x n
    control polyhedron."""
    rows = len(control_points)
    cols = len(control_points[0])
    m = rows - 1
    n = cols - 1

    result = Point3D(0.0, 0.0, 0.0)
    for i in range(rows):
        bu = bernstein(m, i, u)
        for j in range(cols):
            weight = bu * bernstein(n, j, v)
            result = result + control_points[i][j] * weight
    return result

```

Табуляция поверхности (формула 9):

```

def sample_surface(control_points, steps: int):
    """Return a rectangular sampled grid of the Bezier
    surface."""
    return [
        [bezier_surface(control_points, i / steps, j / steps)
         for j in range(steps + 1)]
        for i in range(steps + 1)
    ]

```

Поворот точки – последовательное применение матриц (10), (11) в соответствии с композицией (13):

```

def rotate_point(p: Point3D, angle_x: float, angle_y: float) ->
Point3D:
    """Rotate around X and then Y, matching the two GUI
    controls."""
    ax = math.radians(angle_x)
    ay = math.radians(angle_y)

    # X rotation
    cy = math.cos(ax)
    sy = math.sin(ax)
    y1 = p.y * cy - p.z * sy
    z1 = p.y * sy + p.z * cy
    x1 = p.x

    # Y rotation
    cx = math.cos(ay)
    sx = math.sin(ay)
    x2 = x1 * cx + z1 * sx
    z2 = -x1 * sx + z1 * cx
    return Point3D(x2, y1, z2)

```

Ортогональное проецирование точки на холст (формула 16) и вычисление габаритов сцены (формула 18):

```

def project_orthographic(p: Point3D, width: int, height: int,
scale: float):
    """Simple orthographic projection onto the XY display
plane."""
    return width / 2.0 + p.x * scale, height / 2.0 - p.y * scale

def bounds(points):
    xs = [p.x for p in points]
    ys = [p.y for p in points]
    zs = [p.z for p in points]
    return min(xs), max(xs), min(ys), max(ys), min(zs), max(zs)

```

Автомасштаб (формула 17):

```

def _get_scale(self, rotated_control):
    if not self.auto_scale.get():
        return self.scale.get()
    all_points = list(rotated_control)
    if not all_points:
        return 50
    xmin, xmax, ymin, ymax, zmin, zmax = bounds(all_points)
    span = max(xmax - xmin, ymax - ymin, zmax - zmin, 1.0)
    w = max(self.canvas.wininfo_width(), 400)
    h = max(self.canvas.wininfo_height(), 300)
    return min(w, h) * 0.72 / span

```

Головной метод отрисовки сцены, реализующий конвейер «поворот – табуляция – проекция – вывод»:

```

def redraw(self):
    if not hasattr(self, "canvas"):
        return
    self._read_editor_silent()
    self.canvas.delete("all")
    width = max(self.canvas.wininfo_width(), 400)
    height = max(self.canvas.wininfo_height(), 300)

    # Background grid / axes for orientation.
    self._draw_axes(width, height)

    rx = float(self.angle_x.get())
    ry = float(self.angle_y.get())
    control_rot = [[rotate_point(p, rx, ry) for p in row]
for row in self.control_points]
    flat_control = [p for row in control_rot for p in row]
    scale = self._get_scale(flat_control)

    def project(p):
        return project_orthographic(p, width, height, scale)

    if self.show_control.get():
        self._draw_control_polyhedron(control_rot, project)

```

```

        surface = sample_surface(self.control_points, max(4,
min(60, int(self.surface_steps.get()))))
        surface_rot = [[rotate_point(p, rx, ry) for p in row]
for row in surface]

    if self.show_surface.get():
        self._draw_surface(surface_rot, project)

    # Redraw control points last for clarity.
    if self.show_control.get():
        for i, row in enumerate(control_rot):
            for j, p in enumerate(row):
                x, y = project(p)
                r = 4
                self.canvas.create_oval(x-r, y-r, x+r, y+r,
fill="#ffb74d", outline="")
                self.canvas.create_text(x+8, y-8,
text=f"P{i},{j}", fill="#e8edf2", anchor="w",
font=("TkDefaultFont", 8))

    self.status.set(f"Безье: {self.rows}×{self.cols}
контрольных точек · поворот X={rx:.1f}°, Y={ry:.1f}°")

```

Отрисовка задающего многогранника и сетки поверхности:

```

def _draw_control_polyhedron(self, grid, project):
    # Edges along both parameter directions form the
characteristic/control grid.
    for i in range(len(grid)):
        for j in range(len(grid[i]) - 1):
            a, b = project(grid[i][j]),
project(grid[i][j+1])
            self.canvas.create_line(*a, *b, fill="#8d6e63",
width=1, dash=(3, 3))
        for j in range(len(grid[0])):
            for i in range(len(grid) - 1):
                a, b = project(grid[i][j]),
project(grid[i+1][j])
                self.canvas.create_line(*a, *b, fill="#8d6e63",
width=1, dash=(3, 3))

def _draw_surface(self, grid, project):
    rows, cols = len(grid), len(grid[0])
    # Surface mesh: u-lines and v-lines. The mesh is the
linear approximation of the sampled surface.
    if self.show_grid.get():
        for i in range(rows):
            for j in range(cols - 1):
                a, b = project(grid[i][j]),
project(grid[i][j+1])
                self.canvas.create_line(*a, *b,
fill="#69b7ff", width=1)
            for j in range(cols):
                for i in range(rows - 1):

```

```

        a, b = project(grid[i][j]),
project(grid[i+1][j])
        self.canvas.create_line(*a, *b,
fill="#69b7ff", width=1)
    else:
        # Outline only.
        for edge in (grid[0], grid[-1], [grid[i][0] for i in
range(rows)], [grid[i][-1] for i in range(rows)]):
            pts = [project(p) for p in edge]
            for a, b in zip(pts, pts[1:]):
                self.canvas.create_line(*a, *b,
fill="#69b7ff", width=2)

```

Полный исходный текст программы приведён в приложении А.

3. Интерфейс пользователя

Главное окно программы (рисунок 1) разделено на панель управления (слева) и область визуализации (справа), под которой расположена статусная строка.

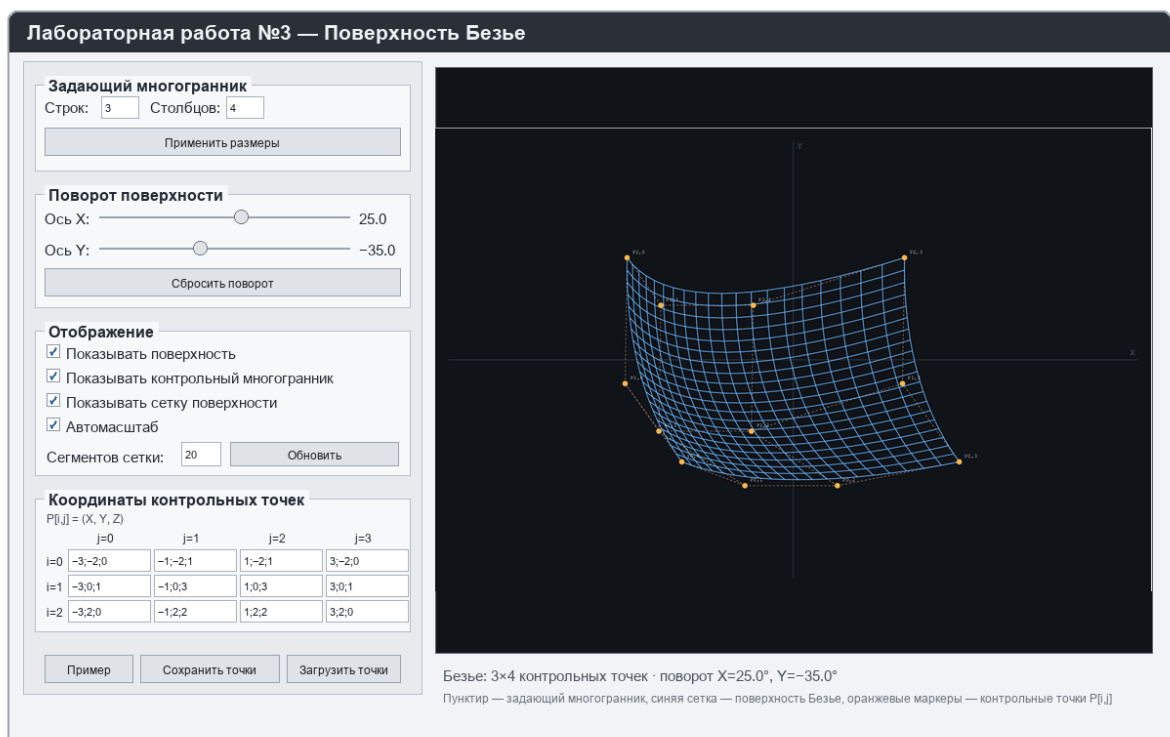


Рисунок 1. Структура главного окна программы

Панель управления построена из пяти групп элементов. Назначение элементов управления приведено в таблице 2.

Таблица 2. Элементы управления программой

Группа	Элемент	Назначение
Задающий многогранник	поля «Строк», «Столбцов» (2...8), кнопка «Применить размеры»	изменение размеров матрицы контрольных точек; существующие координаты сохраняются, новые точки заполняются гладкой синусоидальной заготовкой
Поворот поверхности	ползунки «Ось X», «Ось Y» ($-180...+180^\circ$), кнопка «Сбросить поворот»	плавное изменение углов поворота α и β с немедленной перерисовкой; сброс возвращает значения 25° и -35°
Отображение	флажки «Показывать поверхность», «Показывать контрольный многогранник», «Показывать сетку поверхности», «Автомасштаб»; поле «Сегментов сетки» (4...60) и кнопка «Обновить»	управление составом сцены и плотностью табуляции поверхности N
Координаты контрольных точек	таблица полей ввода в формате «X;Y;Z»	редактирование любой контрольной точки; значения применяются при нажатии Enter и при потере фокуса полем
Файловые операции	кнопки «Пример», «Сохранить точки», «Загрузить точки»	возврат к примеру 3×4 , запись и чтение набора контрольных точек в текстовый файл
Статусная строка	текстовое поле под холстом	размер задающего многогранника, текущие углы поворота, сообщения об ошибках ввода и операциях с файлами

Область визуализации отображает: пунктирные линии коричневого цвета – рёбра задающего многогранника (характеристическую сетку); голубую сетку линий – построенную поверхность Безье; оранжевые маркеры с подписями $P(i, j)$ – контрольные точки; тонкие серые линии – оси X и Y системы координат с подписями. Фон холста тёмный, что обеспечивает высокий контраст всех элементов сцены.

Обмен данными с внешним файлом выполняется в простом текстовом формате: первая строка содержит число строк и столбцов, каждая следующая строка – координаты точек одной строки матрицы в виде x;y;z, разделённые пробелами. Пример файла:

```
3 4
-3;-2;0 -1;-2;1 1;-2;1 3;-2;0
-3;0;1 -1;0;3 1;0;3 3;0;1
-3;2;0 -1;2;2 1;2;2 3;2;0
```

При загрузке проверяются размеры сетки (2...8) и число координат в каждой строке; при неверном вводе в редакторе координат выводится

сообщение в статусной строке, а при ошибке чтения файла – диалоговое окно с описанием причины.

4. Алгоритм работы программы

Алгоритм работы программы состоит из следующих шагов.

- 1) При запуске создаётся окно, панель управления, редактор координат и холст; загружается пример задающего многогранника 3×4 , углы поворота устанавливаются равными $\alpha = 25^\circ$, $\beta = -35^\circ$, число сегментов $N = 20$, и выполняется первая отрисовка сцены.
- 2) Любое действие пользователя (перемещение ползунка, изменение флажка, правка координаты, изменение размеров, загрузка файла) вызывает метод `redraw()`, который полностью перестраивает изображение.
- 3) Координаты контрольных точек считываются из полей редактора, поэтому изменения применяются без дополнительного подтверждения.
- 4) Холст очищается и выводится координатная сетка (оси X и Y).
- 5) Все контрольные точки поворачиваются согласно (13); по габаритам повернутой сетки вычисляется коэффициент масштаба (17).
- 6) Выполняется табуляция поверхности по формуле (9): строится сетка $(N+1) \times (N+1)$ точек, каждая точка которой вычисляется по формуле (3). Полученные точки поворачиваются той же композицией (13).
- 7) Выводится задающий многогранник: отрезки между соседними контрольными точками по строкам и по столбцам матрицы.
- 8) Выводится поверхность: отрезки между соседними точками вдоль параметров u и v . Если флажок «Показывать сетку поверхности» снят, рисуются только четыре граничные кривые (контур поверхности).
- 9) Поверх сетки выводятся маркеры контрольных точек и их подписи $P(i, j)$, а в статусной строке – текущие параметры сцены.

Формально конвейер визуализации можно записать следующим псевдокодом:

```
ПОВОРОТ(alpha, beta, P):  
    для каждой точки p из P:  
        p1 = R_x(alpha) * p          # формула (10)  
        p2 = R_y(beta) * p1         # формула (11)  
    вернуть p2  
  
ПОСТРОИТЬ_СЦЕНУ(control_points, N, alpha, beta):  
    control_rot = ПОВОРОТ(alpha, beta, control_points)  
    k = АВТОМАСШТАБ(control_rot)    # формула (17)  
    grid = табуляция поверхности по (3) и (9)  
    grid_rot = ПОВОРОТ(alpha, beta, grid)  
    вывести оси координат  
    вывести рёбра задающего многогранника (control_rot)  
    вывести сетку поверхности (grid_rot)  
    вывести маркеры и подписи контрольных точек
```

Вычислительная сложность одного кадра определяется табуляцией поверхности: $O(N^2 \cdot N_r \cdot N_c)$ операций умножения, что при предельных значениях ($N = 60$, сетка 8×8) составляет около 240 тысяч операций (3721 точка поверхности на 64 контрольные точки) и не создаёт заметной задержки при перемещении ползунков.

5. Тестирование

Визуальные проверки. На рисунках 2–11 приведены результаты работы программы (копии содержимого области визуализации) для различных задающих многогранников и углов разворота сцены.

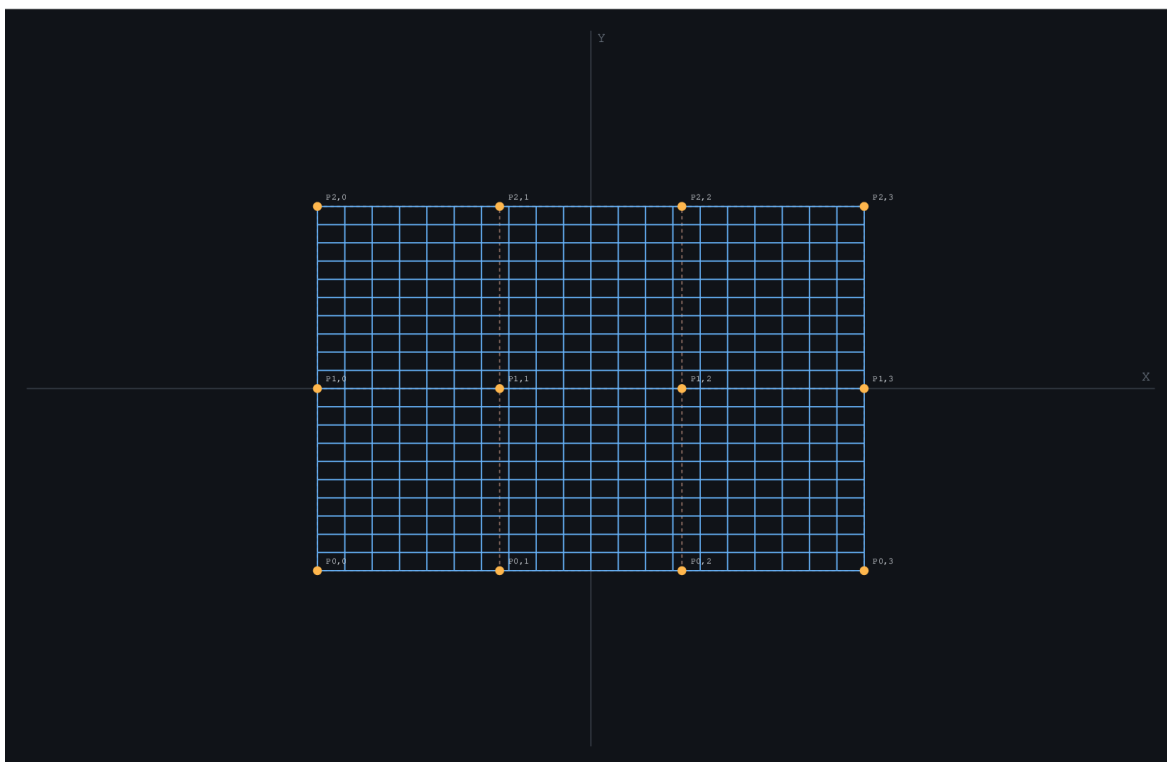


Рисунок 2. Базовое положение поверхности: $\alpha = 0^\circ$, $\beta = 0^\circ$, задающий многогранник 3×4

В базовом положении (рисунок 2) поверхность построена по сетке 3×4 и проходит через четыре угловые контрольные точки, полностью располагаясь внутри задающего многогранника, показанного пунктиром.

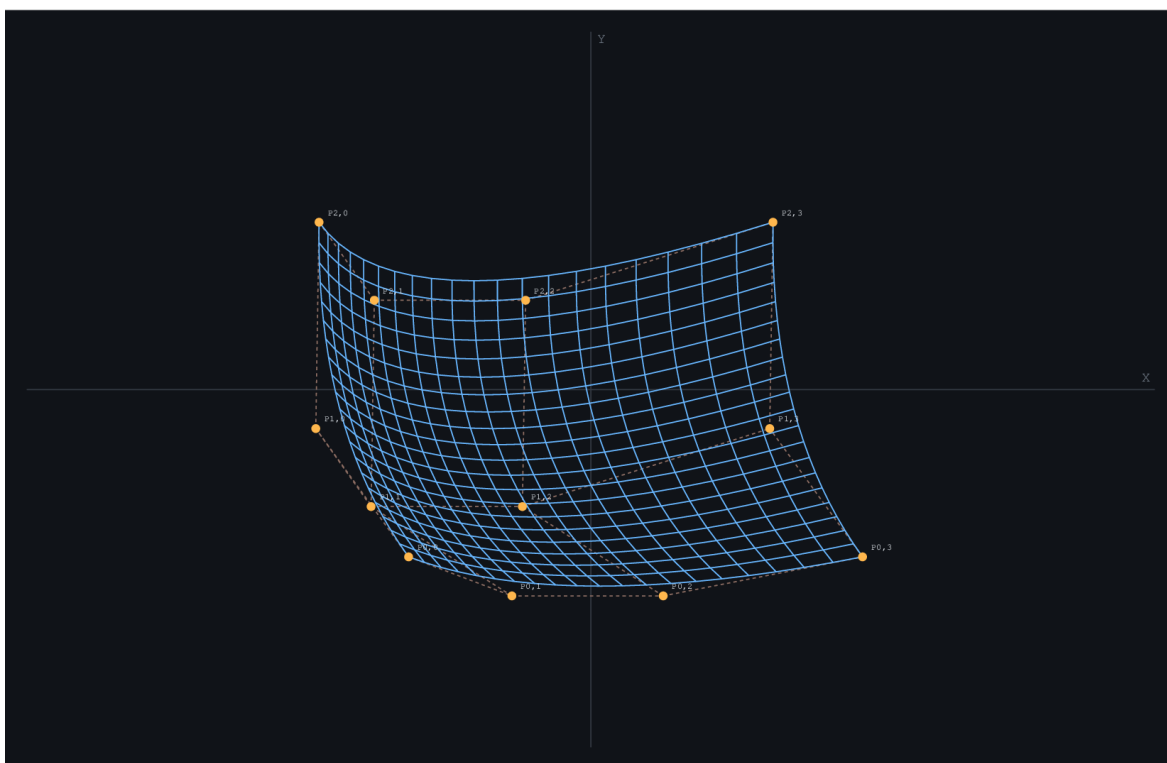


Рисунок 3. Исходный вид программы: $\alpha = 25^\circ$, $\beta = -35^\circ$, задающий многогранник 3×4

Исходный вид программы (рисунок 3) устанавливается при запуске: пространственный разворот сцены позволяет оценить рельеф поверхности.

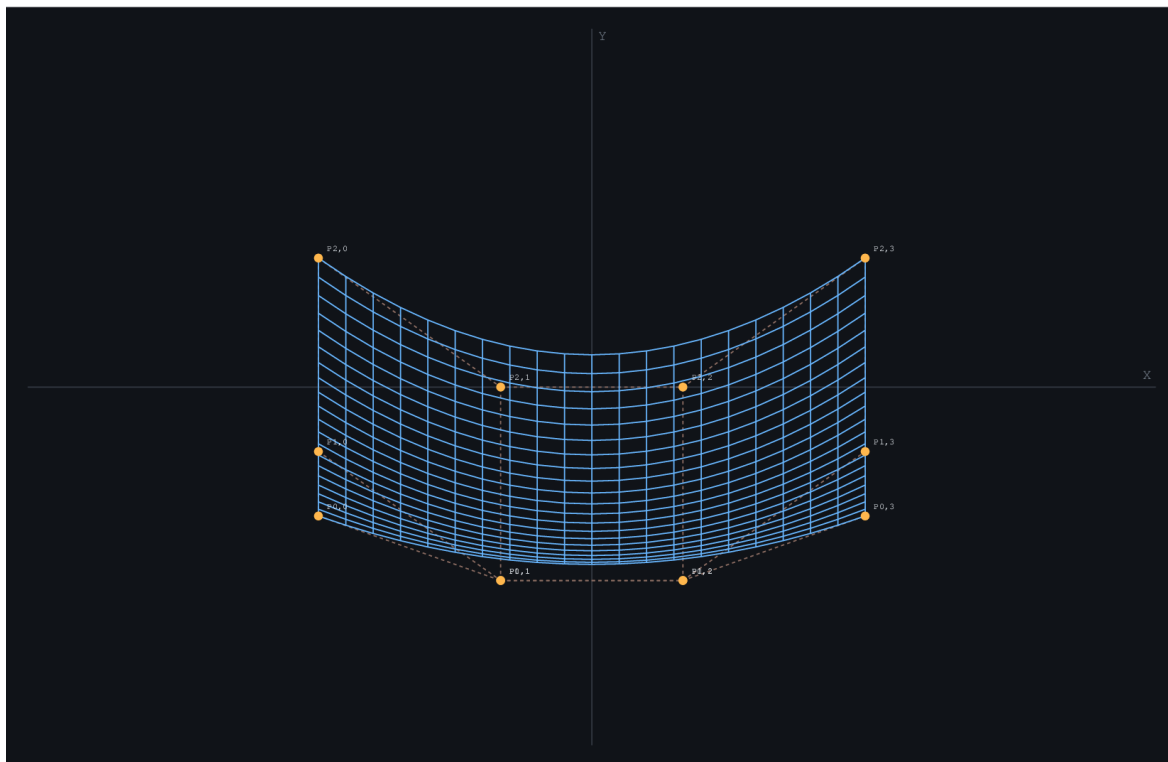


Рисунок 4. Поворот вокруг оси X на угол $\alpha = 45^\circ$

При повороте вокруг оси X (рисунок 4) поверхность и задающий многогранник разворачиваются относительно горизонтальной оси, согласованность их взаимного расположения сохраняется.

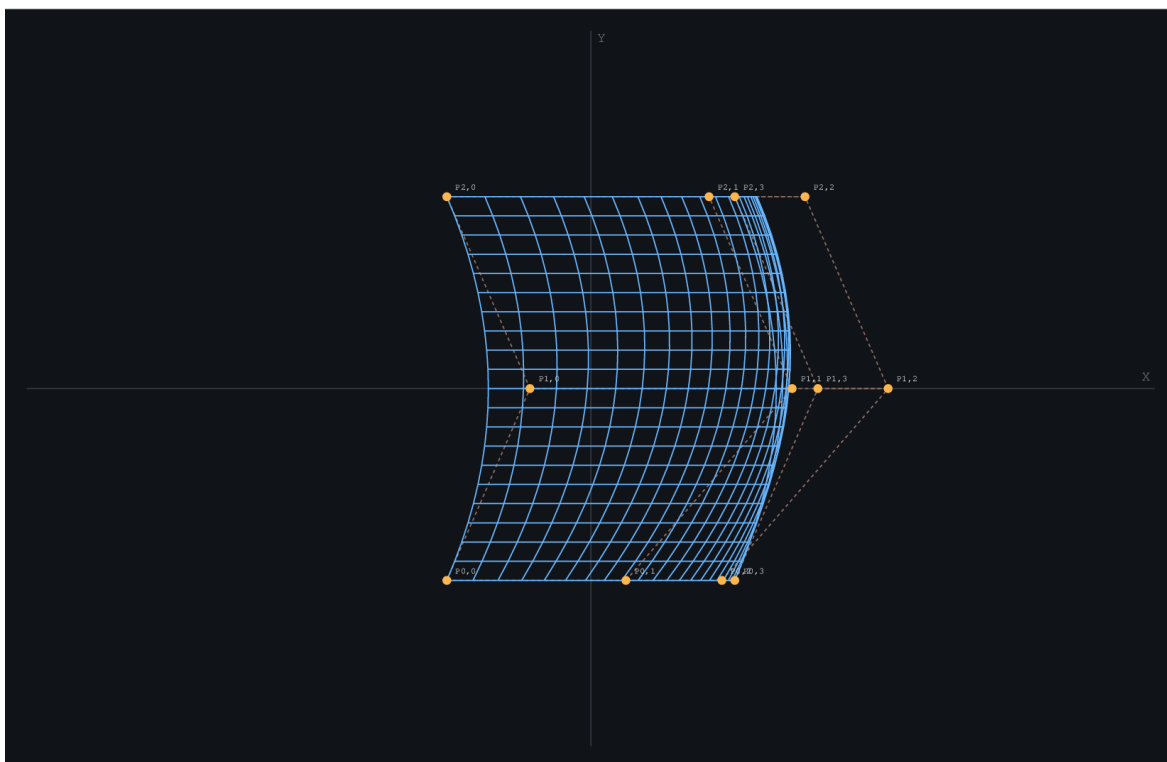


Рисунок 5. Поворот вокруг оси Y на угол $\beta = 60^\circ$

Поворот вокруг оси Y (рисунок 5) демонстрирует вращение сцены вокруг вертикальной оси; изображение перестраивается без искажения пропорций благодаря единому коэффициенту масштаба по обеим осям.

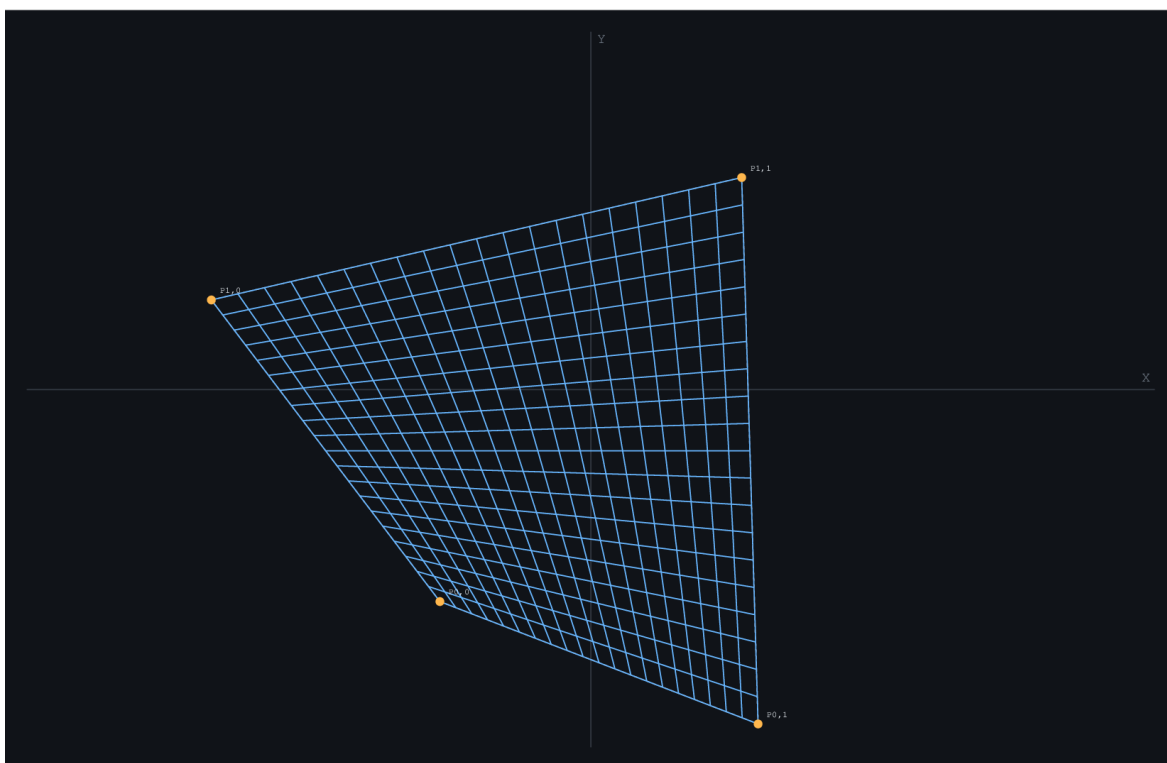


Рисунок 6. Задающий многогранник 2×2 : билинейная поверхность (формула 8)

Для сетки 2×2 (рисунок 6) поверхность вырождается в билинейную (линейчатую, «седловую»), построенную по четырём угловым точкам; результат совпадает с формулой (8).

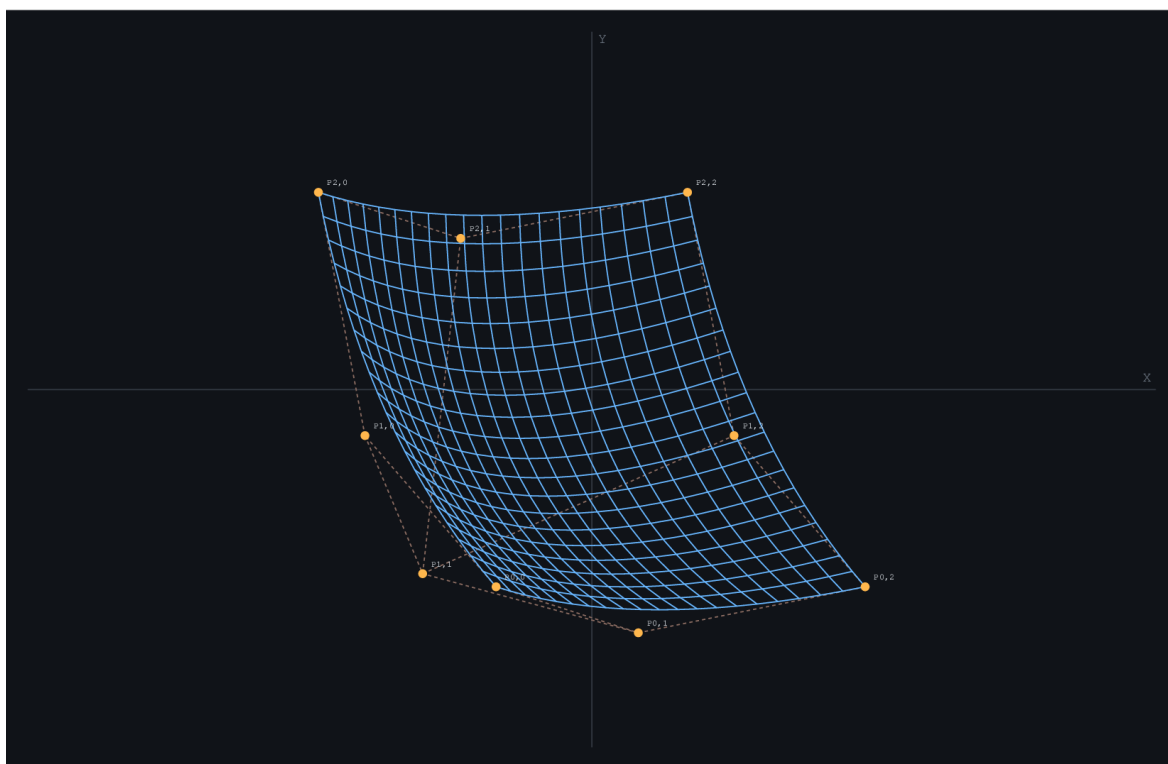


Рисунок 7. Задающий многогранник 3×3 : поверхность с внутренней контрольной точкой

Сетка 3×3 (рисунок 7) содержит внутреннюю контрольную точку, задающую локальный подъём поверхности: поверхность интерполирует только угловые точки и приближается к центральной точке.

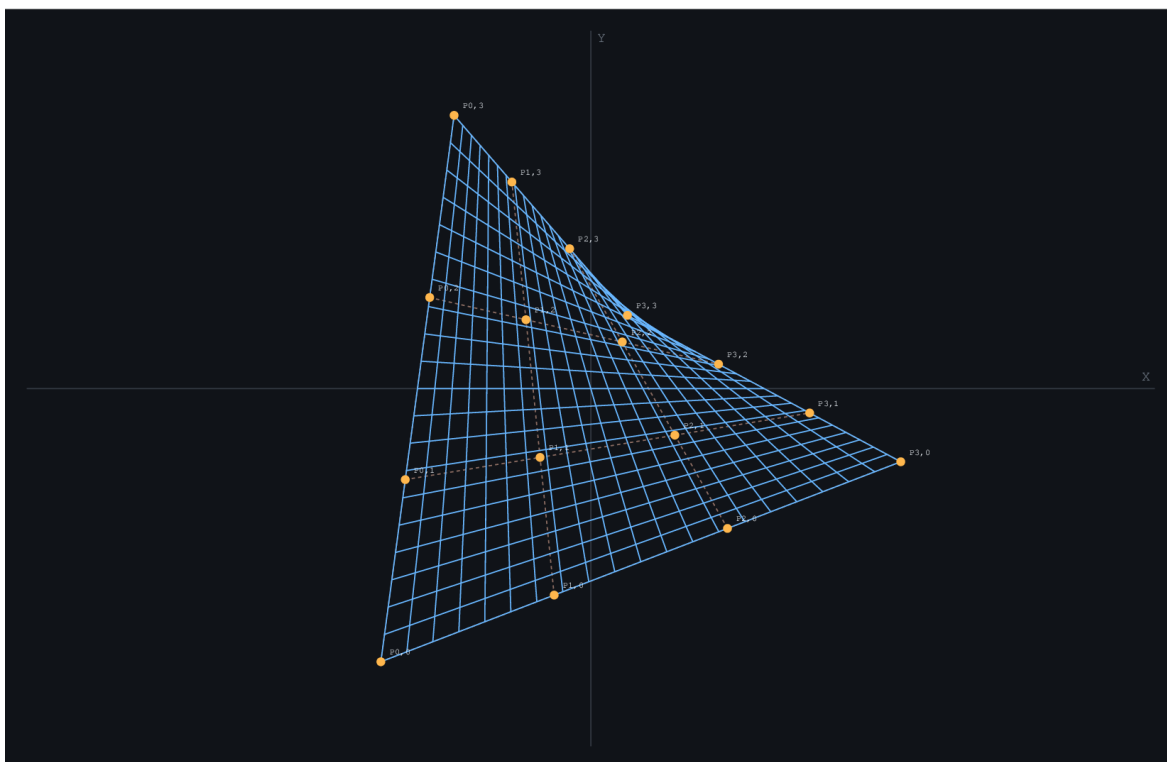


Рисунок 8. Задающий многогранник 4×4 (гиперболический параболоид)

Квадратичная сетка 4×4 (рисунок 8) формирует седлообразную поверхность, характерную для гиперболического параболоида.

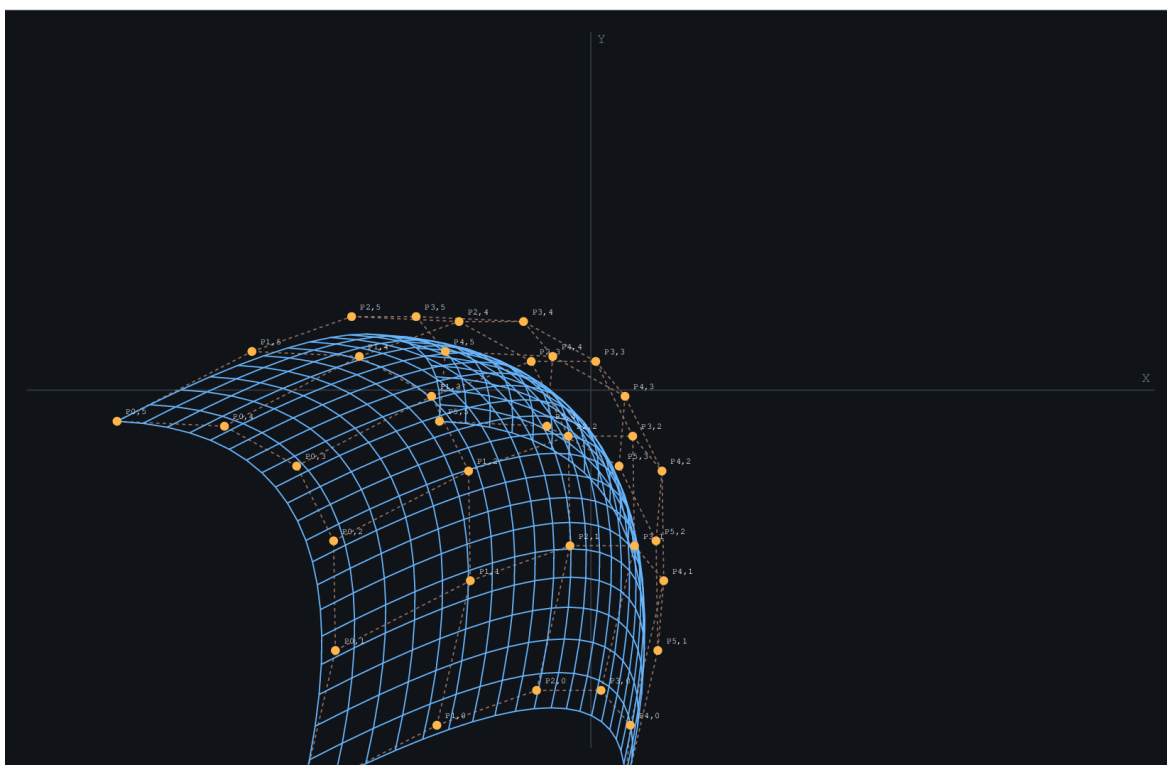


Рисунок 9. Задающий многогранник 6×6 (поверхность с выпуклым рельефом)

На рисунке 9 показана поверхность по сетке 6×6 – программа одинаково работает с любым числом контрольных точек в разрешённом диапазоне $2 \dots 8$ по каждой стороне.

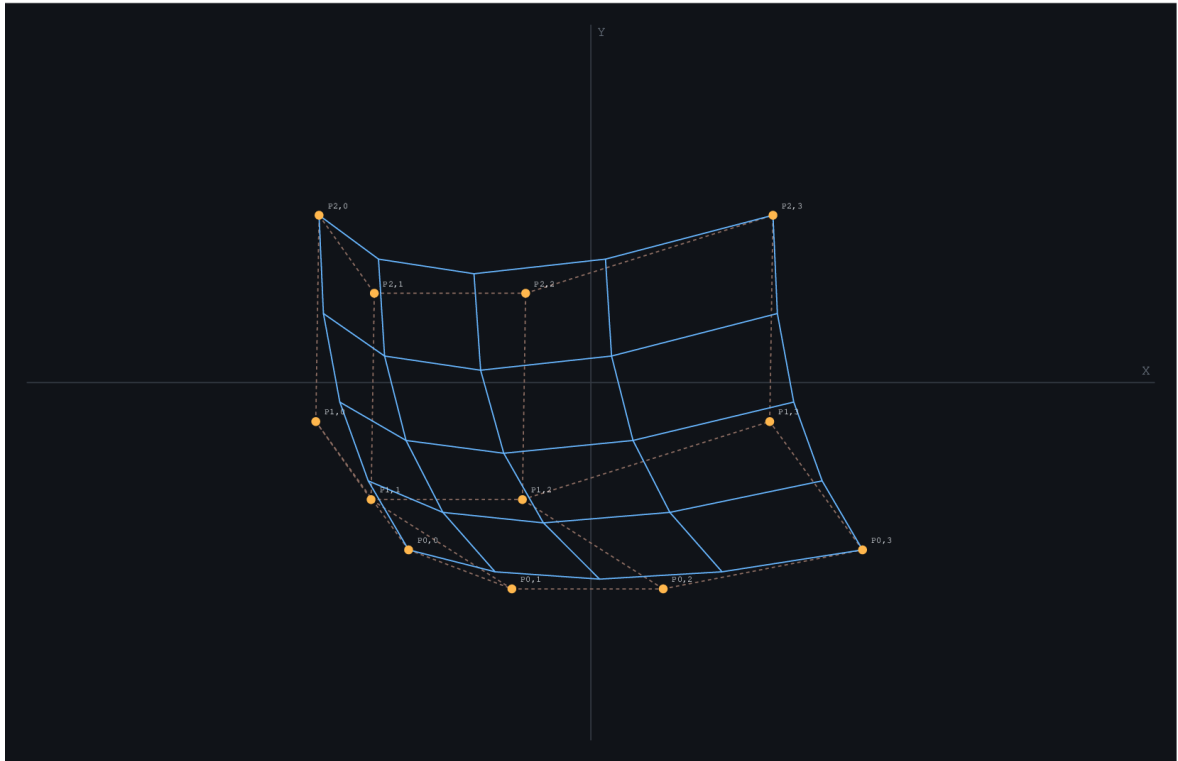


Рисунок 10. Влияние плотности табуляции: $N = 4$ сегмента (сравните с рисунком 3, где $N = 20$)

Уменьшение числа сегментов табуляции до $N = 4$ (рисунок 10) делает линейную аппроксимацию поверхности заметной: каркас становится «гранёным». Это подтверждает вывод о порядке погрешности $O(1/N^2)$, сделанный в разделе 3 математической части.

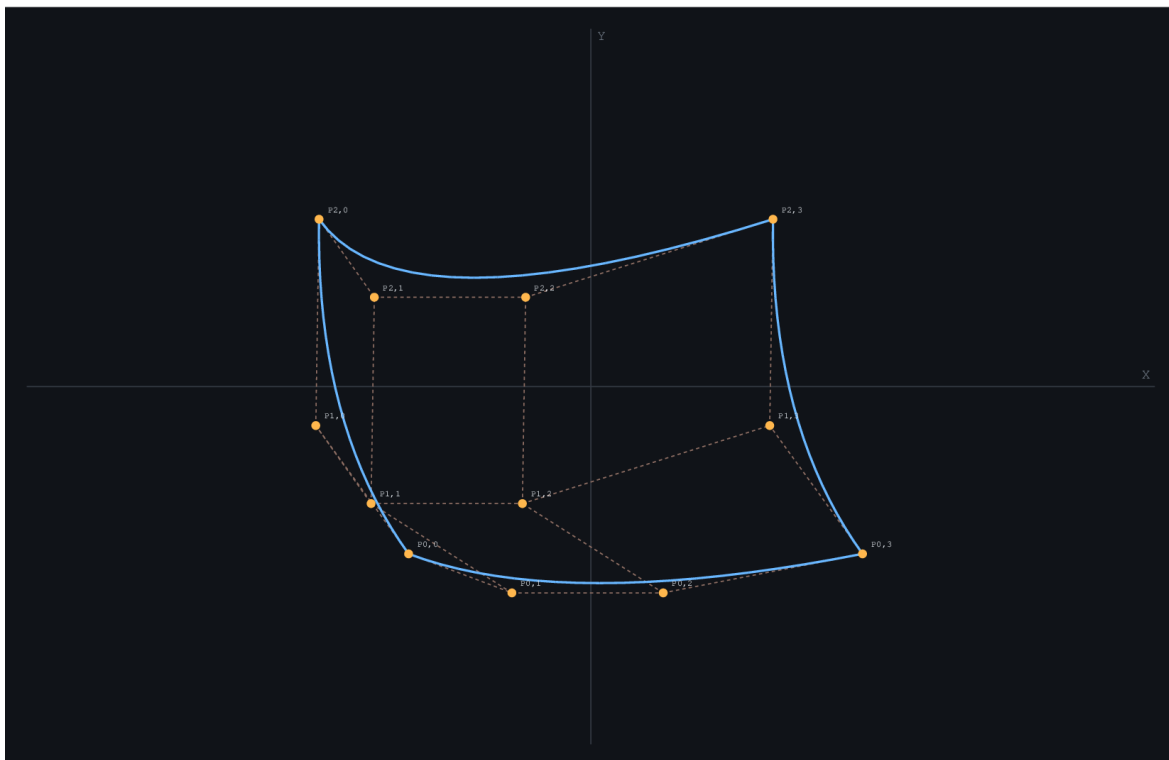


Рисунок 11. Режим отображения только контура поверхности (сетка выключена)

При отключённом флажке «Показывать сетку поверхности» (рисунок 11) выводятся только четыре граничные кривые поверхности, что соответствует режиму контурного представления.

Обработка ошибочных ситуаций также проверена: ввод координат в неверном формате сопровождается сообщением в статусной строке, задание размеров вне диапазона 2...8 – диалоговым окном, загрузка файла с неверным числом точек – сообщением об ошибке чтения.

ВЫВОД

В ходе выполнения лабораторной работы изучен математический аппарат формирования параметрических поверхностей в компьютерной графике: базисные полиномы Бернштейна и построенная на их основе поверхность Безье как тензорное произведение двух семейств аппроксимирующих кривых. Рассмотрены свойства поверхности – интерполяция угловых точек, принадлежность выпуклой оболочке задающего многогранника, тензорная делимость формулы, вырождение в билинейную поверхность при сетке 2×2 .

Освоены пространственные аффинные преобразования трёхмерных объектов: построены матрицы поворота вокруг осей X и Y, выполнена их композиция для последовательного разворота сцены, показано сохранение метрических свойств изображения (длин и углов). Реализовано ортогональное проецирование трёхмерной сцены на плоскость визуализации с автоматическим выбором коэффициента масштаба по габаритам повернутого задающего многогранника.

Разработан программный продукт на языке Python, обеспечивающий: построение поверхности Безье для задающего многогранника произвольного размера от 2×2 до 8×8 ; интерактивное редактирование координат контрольных точек; отображение поверхности, характеристической сетки и маркеров контрольных точек; плавный поворот сцены вокруг осей X и Y; изменение плотности табуляции поверхности; сохранение и загрузку исходных данных. Программа использует только стандартную библиотеку языка (сторонние и специализированные графические библиотеки, в том числе OpenGL, не применялись) и дополнительно упакована в контейнер с виртуальным дисплеем Xvfb и веб-клиентом noVNC для демонстрации интерфейса через браузер.

Корректность реализации подтверждена четырьмя модульными тестами (все пройдены) и численными проверками: максимальное отклонение суммы базисных полиномов от единицы составило $2,2 \cdot 10^{-16}$; расхождение реализованного поворота с произведением матриц – $4,4 \cdot 10^{-16}$; при сетке 2×2 поверхность совпала с билинейной формулой с нулевым отклонением; координаты поверхности не выходят за габариты задающего многогранника; поворот сохраняет длины отрезков. Визуальные проверки для сеток $2 \times 2 \dots 6 \times 6$, различных углов разворота и плотностей табуляции подтвердили устойчивую и наглядную работу программы. Поставленное задание выполнено полностью.